Optimización de un modelo

M6: Aprendizaje de máquina supervisado

|**AE7**: Aplicar técnicas de optimización de un modelo de aprendizaje para la mejora de su desempeño acorde a las buenas prácticas de la industria.

alkemy

Introducción👩‍💻

En el desarrollo de modelos de aprendizaje automático, obtener buenos resultados no solo depende del algoritmo elegido, sino también de los mecanismos utilizados para optimizar su desempeño. Estos mecanismos incluyen desde la ingeniería de características hasta el ajuste fino de hiperparámetros, todos aplicados bajo buenas prácticas propias de la industria. En esta lección abordaremos herramientas clave para potenciar modelos predictivos mediante técnicas consolidadas y modernas.

Comprender estos elementos es fundamental para asegurar que los modelos generalicen adecuadamente sobre datos nuevos. Muchas veces, los errores que se presentan en entornos productivos provienen de un proceso de entrenamiento que no consideró adecuadamente estos mecanismos. Optimizar modelos no es un lujo, sino una necesidad operativa.

Este manual está orientado tanto a quienes se inician en la ciencia de datos como a profesionales que buscan sistematizar su práctica según estándares industriales. Se pretende presentar los conceptos de forma clara, acompañados de ejemplos prácticos y contrastes comparativos cuando sea pertinente.

Al finalizar, contarás con una guía práctica para aplicar estas técnicas en proyectos reales, con énfasis en la biblioteca Scikit-learn, herramienta ampliamente utilizada por su flexibilidad y compatibilidad con todo el ecosistema de ciencia de datos en Python.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Reconocer y aplicar mecanismos de mejora del desempeño en modelos predictivos.
- Diseñar un flujo de trabajo que incluya Feature Engineering.
- Ajustar hiperparámetros usando herramientas como GridSearchCV.
- Aplicar regularización para evitar sobreajuste.
- Balancear datos desbalanceados usando técnicas apropiadas.
- Utilizar Scikit-learn para automatizar procesos de optimización de modelos.

Mecanismos para mejorar el desempeño de un algoritmo

Mejorar un modelo predictivo no solo consiste en seleccionar el algoritmo más sofisticado. Las prácticas habituales incluyen el preprocesamiento de datos, el diseño de nuevas características, la selección y ajuste de hiperparámetros, el uso de regularización y la validación cruzada. Todas estas técnicas tienen un impacto directo en la capacidad de generalización del modelo.

En muchos casos, una buena preparación de los datos puede tener un efecto más relevante que cambiar de algoritmo. Por ejemplo, tratar valores atípicos o normalizar los datos puede hacer que un modelo lineal tenga un rendimiento similar al de modelos más complejos.

Otro mecanismo importante es la reducción de dimensionalidad. Técnicas como PCA (Análisis de Componentes Principales) ayudan a eliminar redundancia en los datos, mejorando la eficiencia del modelo y reduciendo el riesgo de sobreajuste.

La selección adecuada del método de validación también mejora el rendimiento. Validación cruzada estratificada o k-fold permiten estimar de forma más robusta el desempeño fuera de la muestra.

Finalmente, es clave automatizar y documentar todos estos procesos mediante pipelines reproducibles. Esto asegura que las mejoras sean sistemáticas y auditables.

¿Qué es Feature Engineering?

El Feature Engineering o ingeniería de características es el proceso de seleccionar, transformar o crear variables predictoras (features) a partir de datos en bruto. Este proceso es esencial para mejorar la capacidad de aprendizaje de los algoritmos y, por ende, el desempeño del modelo final. Un modelo sencillo con buenas variables puede superar con creces a uno complejo entrenado con datos irrelevantes o mal representados.

Uno de los pasos fundamentales en el Feature Engineering es la selección de variables relevantes. No todas las columnas de un conjunto de datos aportan información útil para el modelo; algunas pueden incluso introducir ruido.

Seleccionar cuidadosamente las características más representativas ayuda a reducir la complejidad y mejora la interpretabilidad del modelo.

Otro componente clave es la transformación de variables. Esto incluye desde operaciones matemáticas simples (como logaritmos o potencias) hasta técnicas más complejas como la codificación de variables categóricas. Por ejemplo, técnicas como One Hot Encoding o Label Encoding permiten convertir datos categóricos en un formato numérico comprensible por los algoritmos.

También es común la creación de nuevas variables mediante la combinación o interacción de otras. Por ejemplo, en un modelo que analiza préstamos bancarios, una variable como "ingreso por miembro del hogar" (ingreso total dividido entre número de personas) puede ofrecer más valor predictivo que usar esas dos variables por separado.

Finalmente, es importante escalar o normalizar las variables numéricas, especialmente si se usarán algoritmos sensibles a la magnitud de los datos, como los modelos lineales o de distancia (SVM, KNN). El Feature Engineering requiere creatividad, conocimiento del dominio y prueba continua, y su correcta aplicación marca la diferencia entre un modelo mediocre y uno excelente.

¿Qué es un Hiperparámetro?

Un **hiperparámetro** es un valor que se configura antes del proceso de entrenamiento de un modelo de aprendizaje automático y que influye en cómo se entrena ese modelo. A diferencia de los **parámetros del modelo**, que se aprenden directamente de los datos durante el entrenamiento (como los coeficientes de una regresión), los hiperparámetros son establecidos manualmente o mediante técnicas de búsqueda automatizada.

Por ejemplo, si usamos un árbol de decisión, podemos definir hiperparámetros como max_depth (profundidad máxima del árbol), min_samples_split (mínimo número de muestras para dividir un nodo), o criterion (la función para medir la calidad de una división). Estos valores no se ajustan solos; deben probarse distintas combinaciones para encontrar los que ofrecen el mejor rendimiento.

La selección adecuada de hiperparámetros es crítica para controlar el equilibrio entre **subajuste (underfitting)** y **sobreajuste (overfitting)**. Si un árbol tiene una profundidad muy baja (max_depth=1), es probable que no aprenda correctamente; si es demasiado profundo, podría aprender hasta el ruido del conjunto de entrenamiento, perdiendo capacidad de generalización.

Para automatizar la selección de hiperparámetros se utilizan técnicas como GridSearchCV o RandomizedSearchCV. Estas herramientas integradas en Scikit-learn permiten evaluar múltiples combinaciones mediante validación cruzada y seleccionar la mejor configuración de forma sistemática.

Aquí un ejemplo práctico con GridSearchCV aplicado a un modelo de árbol de decisión:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.tree import DecisionTreeClassifier

# Cargar datos de ejemplo
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)

# Definir el modelo base
dt = DecisionTreeClassifier(random_state=42)

# Definir la grilla de hiperparámetros
param_grid = {
    'max_depth': [2, 4, 6],
    'min_samples_split': [2, 5, 10],
    'criterion': ['gini', 'entropy']
}

# Aplicar GridSearchCV
grid_search = GridSearchCV(estimator=dt, param_grid=param_grid, cv=5)
grid_search.fit(X_train, y_train)

# Mostrar los mejores parámetros encontrados
print("Mejores hiperparámetros encontrados:")
print(grid_search.best_params_)

# Evaluar el modelo optimizado
print("Precisión en test:", grid_search.score(X_test, y_test))
```

Este ejemplo demuestra cómo una correcta búsqueda de hiperparámetros puede mejorar el rendimiento de un modelo de manera reproducible. Es una práctica estándar en entornos profesionales y clave para la producción de modelos robustos.

¿Qué es la Regularización?

La **regularización** es una técnica crucial en el aprendizaje automático que se utiliza para evitar el **sobreajuste** (overfitting), una situación en la que el modelo

se ajusta demasiado bien a los datos de entrenamiento, pero falla al generalizar a datos nuevos. En términos simples, la regularización penaliza la complejidad del modelo, obligándolo a mantener sus parámetros bajo control.

Existen varios tipos de regularización, pero los más comunes son **L1 (Lasso)**, **L2 (Ridge)** y la combinación de ambas conocida como **Elastic Net**. La regularización L1 tiende a reducir a cero los coeficientes menos relevantes, por lo que también actúa como un método de selección de variables. En cambio, L2 distribuye la penalización entre todos los coeficientes, reduciendo su magnitud pero sin eliminarlos por completo.

La elección entre L1, L2 o Elastic Net depende del problema. Si se sospecha que solo algunas variables son útiles, L1 es ideal. Si hay muchas variables ligeramente relevantes, L2 puede ser mejor. Elastic Net es útil cuando se tienen muchas variables correlacionadas y se quiere un balance entre las ventajas de ambas penalizaciones.

En Scikit-learn, existen modelos que integran la regularización de forma nativa, como Lasso, Ridge o ElasticNet. Estos modelos permiten ajustar un hiperparámetro clave: **alpha**, que controla la intensidad de la penalización. Cuanto mayor sea el valor de alpha, más se reducirá la magnitud de los coeficientes, lo que puede ayudar a evitar el sobreajuste pero también puede inducir subajuste si se exagera.

A continuación se muestra un ejemplo práctico de cómo aplicar regularización con regresión Lasso en Scikit-learn:

```
from sklearn.linear_model import Lasso
from sklearn.datasets import load_boston
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Cargar datos de ejemplo
X, y = load_boston(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)

# Crear modelo Lasso con regularización
lasso = Lasso(alpha=1.0)
lasso.fit(X_train, y_train)

# Evaluación del modelo
y_pred = lasso.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print("Error cuadrático medio con Lasso:", mse)
```

Este ejemplo ilustra cómo la regularización L1 puede ser implementada fácilmente para reducir la complejidad del modelo. Modificando el valor de alpha, puedes experimentar diferentes grados de penalización para observar su impacto en el desempeño. Esta práctica es fundamental en entornos donde se busca un equilibrio entre precisión y generalización.

Optimización de parámetros

La optimización de parámetros es una etapa clave en el ciclo de vida de un modelo de aprendizaje automático, ya que permite mejorar su rendimiento al encontrar la mejor combinación de hiperparámetros. Estos no se aprenden directamente de los datos, por lo que deben ajustarse mediante experimentación y validación. Un mal ajuste puede conducir a modelos que generalizan mal, ya sea por subajuste o sobreajuste.

Una de las estrategias más utilizadas para optimizar hiperparámetros es la **búsqueda en grilla (Grid Search)**. Este método prueba todas las combinaciones posibles de un conjunto definido de valores. Aunque puede ser costoso computacionalmente, garantiza una evaluación exhaustiva y es especialmente útil cuando se tiene un espacio de búsqueda acotado y bien definido.

Otra alternativa es la **búsqueda aleatoria (Random Search)**, que selecciona combinaciones aleatorias de hiperparámetros dentro de los rangos establecidos. Este enfoque es más eficiente en problemas con muchos hiperparámetros o espacios grandes de búsqueda, y suele encontrar buenas soluciones con menos evaluaciones que Grid Search.

Para modelos complejos, se pueden emplear métodos más avanzados como la **optimización bayesiana**, que modela el rendimiento del modelo como una función probabilística y escoge nuevas combinaciones de hiperparámetros donde es más probable encontrar mejoras. Sin embargo, estos métodos requieren bibliotecas adicionales como Optuna o BayesianOptimization.

A continuación se presenta un ejemplo usando GridSearchCV de Scikit-learn con un modelo de Random Forest:

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.datasets import load_iris

# Cargar conjunto de datos
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)

# Definir el modelo base
clf = RandomForestClassifier(random_state=42)

# Definir la grilla de hiperparámetros
param_grid = {
    'n_estimators': [50, 100, 150],
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 4]
}

# Configurar GridSearchCV
grid_search = GridSearchCV(estimator=clf, param_grid=param_grid, cv=5,
                           scoring='accuracy')
grid_search.fit(X_train, y_train)

# Resultados
print("Mejores parámetros:", grid_search.best_params_)
print("Precisión en test:", grid_search.score(X_test, y_test))
```

Grilla de parámetros

La **grilla de parámetros** es una estructura que define el conjunto de valores que serán evaluados durante el proceso de optimización de hiperparámetros. Esta estrategia es implementada frecuentemente en la técnica de Grid Search, en la cual se combinan todos los valores posibles especificados en la grilla y se evalúa el desempeño del modelo con cada una de esas combinaciones.

El objetivo de una grilla de parámetros es permitir una exploración sistemática del espacio de búsqueda. Por ejemplo, si definimos tres posibles valores para el número de árboles (n_estimators) y tres para la profundidad máxima (max_depth), Grid Search probará las nueve combinaciones posibles. Aunque este método garantiza una evaluación completa, también puede ser costoso computacionalmente si se amplía el número de valores o hiperparámetros.

Una ventaja importante de la grilla de parámetros es su reproducibilidad. Al definir explícitamente cada combinación, los experimentos pueden ser replicados

fácilmente, lo que resulta fundamental en contextos donde la trazabilidad y la validación de modelos son prioritarias. También permite registrar el rendimiento asociado a cada configuración, lo que facilita el análisis comparativo.

Sin embargo, su principal desventaja es la **ineficiencia computacional** cuando el número de combinaciones crece exponencialmente. Por esta razón, se recomienda utilizar grillas de tamaño manejable o, en su defecto, aplicar alternativas como búsqueda aleatoria (RandomizedSearchCV) o técnicas bayesianas más eficientes.

Aquí un ejemplo que muestra cómo construir una grilla de parámetros con GridSearchCV para un clasificador de soporte vectorial (SVM):

```python
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.datasets import load_digits

# Cargar datos de ejemplo
X, y = load_digits(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)

# Definir el modelo
svc = SVC()

# Definir la grilla
param_grid = {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 'auto']
}

# Ejecutar GridSearchCV
grid = GridSearchCV(estimator=svc, param_grid=param_grid, cv=5, scoring='accuracy')
grid.fit(X_train, y_train)

# Resultados
print("Mejor combinación de parámetros:", grid.best_params_)
print("Precisión en test:", grid.score(X_test, y_test))
```

Este ejemplo muestra cómo una grilla bien diseñada permite probar diversas configuraciones del modelo y encontrar la que ofrece mejor rendimiento. La clave está en definir una grilla equilibrada que contemple suficiente variabilidad sin sacrificar eficiencia computacional. Esto es especialmente útil cuando se integran los modelos en sistemas de producción donde se exige robustez y trazabilidad.

Balanceo de datos

En el aprendizaje automático supervisado, el **balanceo de datos** es esencial cuando se trabaja con conjuntos de datos desbalanceados, es decir, cuando las clases no están representadas de forma equitativa. Esto es común en casos de detección de fraudes, diagnóstico médico o análisis de abandono de clientes, donde una clase puede estar presente en el 90% de los casos y otra solo en el 10%. Los modelos tienden a favorecer la clase mayoritaria, lo que afecta negativamente la calidad de las predicciones.

Uno de los efectos más problemáticos del desbalanceo es que un modelo puede mostrar una alta precisión global, pero en realidad no estar capturando correctamente la clase minoritaria. Por ejemplo, si solo el 5% de los datos corresponde a fraudes, un modelo que prediga "no fraude" en todos los casos tendrá 95% de precisión, pero no servirá para identificar fraudes reales. Por eso, se deben aplicar estrategias específicas para balancear el conjunto de entrenamiento.

Existen tres enfoques principales para el balanceo: **submuestreo**, **sobremuestreo** y **ponderación de clases**. El submuestreo reduce el número de muestras de la clase mayoritaria, mientras que el sobremuestreo aumenta las muestras de la clase minoritaria (replicando datos o generando datos sintéticos con técnicas como SMOTE). Por su parte, la ponderación ajusta la función de pérdida para que penalice más los errores cometidos sobre la clase minoritaria.

Scikit-learn facilita la implementación de estas estrategias. Algunos modelos como LogisticRegression, RandomForestClassifier o SVC permiten utilizar el parámetro class_weight='balanced' para asignar pesos automáticos inversamente proporcionales a la frecuencia de las clases. Esta es una forma simple y efectiva de mejorar el rendimiento del modelo sin alterar el conjunto de datos.

Aquí tienes un ejemplo práctico con regresión logística:

```python
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Crear un conjunto de datos desbalanceado
X, y = make_classification(n_samples=1000, n_classes=2, weights=[0.9, 0.1],
                           n_informative=3, n_redundant=0, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)

# Modelo con ponderación automática de clases
model = LogisticRegression(class_weight='balanced')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Evaluar resultados
print(classification_report(y_test, y_pred))
```

Implementación con la librería Scikit-learn

La librería **Scikit-learn** es una de las herramientas más utilizadas en Python para la creación y optimización de modelos de aprendizaje automático. Su diseño simple y coherente permite construir flujos de trabajo robustos que abarcan desde el preprocesamiento de datos hasta la validación y evaluación de modelos. Gracias a su compatibilidad con otras bibliotecas del ecosistema Python, como NumPy, pandas y Matplotlib, es una opción versátil y poderosa tanto para tareas experimentales como productivas.

Una de las principales ventajas de Scikit-learn es su enfoque modular. Todos los estimadores comparten una API común, lo que permite construir **pipelines** en los que se integran transformadores y modelos de forma encadenada. Esto facilita la organización del código, mejora la reproducibilidad y evita la fuga de datos durante las fases de entrenamiento y validación cruzada.

En Scikit-learn se pueden aplicar fácilmente técnicas de regularización, ajuste de hiperparámetros, selección de características, validación cruzada y balanceo de clases. Además, incluye implementaciones eficientes de algoritmos como regresión logística, árboles de decisión, SVM, bosques aleatorios, k-vecinos, entre otros. Todo esto con documentación clara y ejemplos accesibles.

La integración de GridSearchCV y RandomizedSearchCV permite optimizar hiperparámetros directamente sobre un pipeline, combinando preprocesamiento y modelo en un solo objeto. Esto no solo mejora la legibilidad, sino que reduce errores comunes al ajustar manualmente etapas por separado. También es posible evaluar el rendimiento con múltiples métricas y seleccionar el mejor modelo según la métrica más relevante para el negocio.

A continuación se presenta un ejemplo de pipeline completo con escalado, balanceo de clases y ajuste de hiperparámetros para una regresión logística:

```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.datasets import load_breast_cancer

# Cargar conjunto de datos
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.3, random_state=42)

# Crear pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LogisticRegression(class_weight='balanced', max_iter=1000))
])

# Definir grilla de parámetros
param_grid = {
    'clf__C': [0.1, 1, 10],
    'clf__penalty': ['l1', 'l2'],
    'clf__solver': ['liblinear']
}

# Ajustar modelo
grid = GridSearchCV(pipeline, param_grid, cv=5, scoring='accuracy')
grid.fit(X_train, y_train)

# Resultados
print("Mejores parámetros:", grid.best_params_)
print("Precisión en test:", grid.score(X_test, y_test))
```

Este ejemplo muestra cómo Scikit-learn permite construir un flujo completo y profesional en pocas líneas de código. La correcta implementación de estas herramientas reduce los errores, mejora la productividad del equipo de datos y asegura que los modelos sean confiables para su uso en producción.

Cierre ✍️

A lo largo de este módulo hemos explorado los principales mecanismos para optimizar modelos de aprendizaje automático, desde conceptos teóricos fundamentales hasta su implementación práctica con la biblioteca Scikit-learn. Cada técnica —Feature Engineering, ajuste de hiperparámetros, regularización, balanceo de datos y búsqueda en grilla— cumple un rol esencial para mejorar el desempeño predictivo y garantizar modelos más robustos, escalables y confiables.

Comprender la diferencia entre parámetros e hiperparámetros permite tomar decisiones más informadas al momento de configurar modelos. Asimismo, aplicar técnicas de regularización ayuda a encontrar un equilibrio entre complejidad y generalización, mientras que el uso de métodos de búsqueda sistemática, como Grid Search, asegura un entrenamiento más riguroso y objetivo.

El balanceo de clases es una práctica indispensable en escenarios reales donde los datos suelen estar desbalanceados. Este aspecto, muchas veces subestimado, puede ser decisivo en la efectividad del modelo, especialmente en contextos críticos como salud, seguridad o finanzas. La capacidad de implementar estas soluciones con herramientas accesibles como Scikit-learn potencia el trabajo de los profesionales de datos.

Además, la integración de todo el flujo mediante pipelines facilita la reproducibilidad y trazabilidad, aspectos clave en entornos productivos. La automatización del preprocesamiento, validación y ajuste de parámetros se traduce en eficiencia operativa y calidad del producto final. En un mundo donde los modelos de aprendizaje automático deben evolucionar constantemente, contar con flujos bien estructurados es una ventaja competitiva.

¡Nos vemos de nuevo, más adelante! 👋

Referencias📚

- Towards Data Science. (2020). *Feature engineering for machine learning*. Recuperado de https://towardsdatascience.com/feature-engineering-for-machine-learning-3a5e293a5114
- Scikit-learn Documentation. (2023). *Model evaluation and improvement*. Recuperado de https://scikit-learn.org/stable/model_selection.html
- IBM Developer. (2021). *Hyperparameter tuning with GridSearchCV*. Recuperado de https://developer.ibm.com/articles/hyperparameter-tuning-with-gridsearchcv/
- Analytics Vidhya. (2021, julio). *Regularization techniques (L1 and L2)*. Recuperado el 25 de julio de 2025, de https://www.analyticsvidhya.com/blog/2021/07/prevent-overfitting-using-regularization-techniques/
- Jayawardena, N. (2020, 21 de junio). *Dealing with imbalanced data* [Artículo]. *Towards Data Science*. Recuperado el 25 de julio de 2025, de https://towardsdatascience.com/how-to-deal-with-imbalanced-data-34ab7db9b100
- Scikit-learn. (2023). *Pipeline and FeatureUnion*. Recuperado de https://scikit-learn.org/stable/modules/compose.html

¡Muchas gracias!

Nos vemos en la próxima lección

alkemy